



Security Audit Report

Hydra-Pay

Disclosure

This document contains proprietary information belonging to Entangled Labs (E7D). Duplication, redistribution, or use, in whole or in part, in any form, requires explicit consent from Entangled Labs (E7D). Nonetheless, both the customer Blazar-Labs and Entangled Labs (E7D) are authorized to share this document with the public to demonstrate security compliance and transparency regarding the outcomes of the *Protocol*.

Disclaimer and Scope

This code review shows the state of the project at the time of assessment. This assessment reflects the repository's state at the time of analysis and concentrates on the Aiken on-chain validators for Hydra Pay. Please note that this audit does not cover compiler-generated output and off-chain code, or any components outside the reviewed code.

The observations and recommendations in this report are based solely on the information available during the review period and any changes made after 2025-09-30 fall outside the scope of this report.

We worked to identify security issues, but not every risk may be found. The results here should not be taken as investment advice.

It is highly advised that projects pursue additional independent audits and/or engage in bug bounty programs to further strengthen the security of their protocol.

Executive summary

Hydra-Pay employs solid eUTxO patterns (one-shot minting, receipt-like TokenName binding, unique/tagged outputs, and withdraw-zero to centralize transaction-level checks).

No critical issues were identified.

Overall, the design aligns with best practices and is well-suited for Hydra's operational limits. We recommend keeping the withdraw-zero pattern and planning a migration to *CIP-0112 - Observe Script Type* once available, plus adding simple caps and tests to strengthen DoS resilience and address control.

Code base

Repository: blazar-hydra : <https://github.com/blazarlabs-io/blazar-hydra>

Commit : ed99a650f5d7c838428fe4d339fad935f4480337

Files audited (on-chain):

- `src/onchain/hydra-pay/validators/main.ak` : Entry points and dispatch to checks.
- `src/onchain/hydra-pay/lib/checks.ak` : Core validations for each redeemer type.
- `src/onchain/hydra-pay/lib/types.ak` : Typed datums and redeemers.
- `src/onchain/hydra-pay/lib/utils.ak` : Helpers.

Protocol overview

Actors

- Users:
 - hold *User Funds UTxOs* at the script with `FundsDatum { addr, locked_deposit, funds_type = User { public_key } }`.
 - their balances are spendable inside the Hydra head.
 - `locked_deposit` is the minimum ADA the user's UTxO must retain.
- Merchants:
 - receive *Merchant Funds UTxO* at the script with `FundsDatum { addr, locked_deposit = 0, funds_type = Merchant }`, aggregating all receipts into at most one UTxO per merchant during a head's lifetime.
- Admin:
 - is authorized to couple commits with the running Hydra head and to coordinate lifecycle operations.

Lifecycle

- Users and Merchants:
 - Deposit window (L1): Users can create or augment script UTxOs by depositing funds (minting one control token per Funds UTxO). Multiple deposits per user can exist prior to head open.
 - Fast payments (L2): Inside the Hydra head, users pay merchants quickly and cheaply. Merchant receipts are aggregated into a single Merchant Funds UTxO per merchant.
 - Withdrawals (L2/L1): Merchants typically decommit/withdraw before head close; users may withdraw at any time. At close, a fanout returns remaining user and merchant balances to L1.
- Admin:
 - Pre-open maintenance (L1): Before opening a new head, the admin merges leftover user deposits from the previous cycle with new deposits per user, ensuring one UTxO per user.
 - Open and commit (L1): The admin initializes a Hydra head and commits user Funds UTxOs, optionally after merging per-user deposits to respect head limits. Commit is proven via `CombinedCommit` with hydra input and admin signature.

- Head closure and fanout (L1): The admin coordinates the closing of the head and the fanout of remaining balances back to L1.

Limits

- Commit, CollectCom, and Fanout steps have practical limits. The protocol merges deposits and aggregates merchant funds to keep within Hydra transaction budgets, per the design's conservative bounds.

Design patterns implemented

Parameterized Script Validation

- Implementation:
 - Validator parameters include `admin_key` and `hydra_script`. `validate_combined_commit` gathers input refs from script/admin credentials, checks Hydra input's redeemer equals `HydraCommit { utxos_refs }`, and requires `admin_key` among `extra_signatories`.
- Rationale:
 - Ties commits to the live Hydra head and authorized admin control.
- Mitigated vulnerabilities / attack vectors:
 - Unauthorized protocol actions (e.g., arbitrary commits not bound to the current head).

Forwarding Validation via Withdraw Purpose (Withdraw-Zero)

- Implementation:
 - `spend` handlers for `Commit`, `PartialCommit`, `Merge`, `UserWithdraw`, `MerchantWithdraw` delegate to the `withdraw` purpose with the corresponding combined redeemer (`CombinedCommit`, `CombinedPartialCommit`, `CombinedMerge`, `CombinedWithdraw`).
- Rationale:
 - Prove cross-input invariants once per transaction, not once per input; reduces evaluation budget and avoids repeated checks while still seeing the full transaction.
 - These validations could have been implemented under the spending path, but that would execute once per input; to execute them only once per transaction, the smart contracts developer deliberately use the withdraw-zero staking validator pattern.
- Mitigated vulnerabilities / attack vectors:
 - Enables transaction-level reasoning to better prevent "Double satisfaction" across multiple inputs/outputs.
- Recommendation:
 - When CIP-0112 Observe Script Type is available, migrate these transaction-level checks to an Observe script for cleaner single-execution semantics [link](#).

State Thread Token (STT)-like Control Token

- Implementation:
 - The single control token threads state for each Funds UTxO across transactions; spending and continuing outputs must preserve or burn it under strict rules.

- Rationale:
 - Authenticates “legit” protocol UTxOs and guards datum/state continuity.
- Mitigated vulnerabilities / attack vectors:
 - “Missing UTxO authentication” (foreign UTxOs at the address).
 - Datum spoofing without the authenticated token presence.

Receipt-Like TokenName Derivation

- Implementation:
 - Derive the control token’s TokenName using `utils.output_reference_to_bytestring` (CBOR + blake2b over the input reference).
- Rationale:
 - TokenName becomes a per-UTxO fingerprint, aligning with the “receipt” pattern in Aiken’s docs [link](#).
- Mitigated vulnerabilities / attack vectors:
 - “Other token name” impersonation or arbitrary TokenName choices.
 - Weak binding between token and transaction context.

One-Shot Minting Policy (Enforcing Uniqueness)

- Implementation:
 - Enforce exactly one minted unit under the control policy and require the referenced `OutputReference` to be spent in the same transaction (`validate_minting`).
- Rationale:
 - Guarantees uniqueness and single-use minting bound to a specific UTxO.
- Mitigated vulnerabilities / attack vectors:
 - Multi-minting / inflation of control tokens.
 - Replay or duplicate issuance without consuming the binding reference.

Transaction-Level Spend/Mint Coupling

- Implementation:
 - In `validate_minting`, mint exactly one control token where the TokenName equals `output_reference_to_bytestring(ref)`, and the referenced input is consumed; ensure the script output carries a valid inline `FundsDatum`.
 - In `validate_pay`, if no prior Merchant Funds input exists, require the expected single-unit mint with TokenName bound to the user input reference; otherwise require no mint/burn under the control policy.
- Rationale:
 - Binds the token lifecycle to concrete inputs and datums, coupling Funds UTxO creation and payment aggregation under verifiable constraints.
- Mitigated vulnerabilities / attack vectors:
 - “Other token name” and unauthorized protocol token minting (enforce single TokenName and quantity).
 - “Trust no UTxO” and “Missing UTxO authentication” via binding TokenName to a consumed reference and validating the `FundsDatum`.
 - “Double satisfaction” attempts involving duplicated/forged control tokens.

Unique / Tagged Outputs

- Implementation:
 - `validate_combined_merge` restricts to a single output carrying the control token and requires all inputs share the same datum with value equality checks.
 - `validate_combined_withdraw` recursively proves per-input payouts and that no control tokens remain after burning.
 - `validate_pay` enforces exactly two ordered script outputs with strict value/datum constraints and optional merchant-input coupling.
- Rationale:
 - Tagging outputs and constraining their multiplicity ensures outputs cannot be counted twice across multiple validator executions.
- Mitigated vulnerabilities / attack vectors:
 - “Double satisfaction” of spend conditions via duplicated outputs.

Other Design Decisions

- Datum invariants: Inline datums are enforced for script state; external payouts carry `NoDatum` to minimize UTxO size and avoid datum spoofing.
- No reference scripts on outputs: Prevent accidental fee bloat and preserve predictable output size.
- Allow UTxO Aggregation (Indexer-like merge): Aggregate multiple Funds UTxOs sharing the same datum into a single canonical UTxO with a single control token to reduce UTxO set size (economy and head limits) and provides a canonical state per owner, aligning with the daily head cadence and fanout constraints.

Findings

Severity ratings

- Critical: Enables direct fund loss or unspendable protocol state
 - Major: Unauthorized actions, reward capture, or high-impact logic bypass
 - Medium: DoS/stalling or partial invariant gaps
 - Minor: Low-impact issues, future-proofing, or code clarity
 - Informational: Non-security suggestions, documentation
-

Level	Severity	Findings
5	Critical	0
4	Major	1
3	Medium	1
2	Minor	1
1	Informational	0

ID-401 Insufficient staking credential control (Major)

- Description: CombinedMerge (withdraw): requires only that the output payment credential equals the script; but not the staking credential, enabling arbitrary staking keys on continuing outputs.
(For **Pay** path, checking just payment credential on continuing outputs is acceptable since Hydra L2 addresses do not carry staking credentials).
- Impact: Unpredictable addresses; potential illegitimate staking reward capture.
- Recommendation: Enforce full **Address** equality (payment + staking) for L1 CombinedMerge continuing output (and any other L1 continuing script outputs). Alternatively, ensure a canonical script address (with fixed staking credential) is encoded and matched.

ID-301 Unbounded value cardinality in Pay (Medium)

- Description: - Value relations are checked precisely in **AddFunds** and **Pay**, but there is no cap on distinct assets in **PayInfo.amount** or multi-asset maps in inputs/outputs.
- Impact: Cheap spam/DoS via large cardinalities.
- Recommendation: Add length caps (MAX_ASSETS, MAX_POLICIES) and enforce batching.
Reference:

ID-201 Minor datum constraints (Minor)

- Description: Datum shape is fixed and checked, but no upper bound on **locked_deposit**.
 - Impact: Very Low; potential misconfiguration.
 - Recommendation: Bound **locked_deposit** to a sensible maximum; keep outputs **NoDatum** on payouts. (already enforced no reference scripts on outputs.)
-

